Closure properties of Regular languages

- A closure property is a characteristic of a class of languages (such as regular, context-free, etc.) where applying a specific operation (like union, intersection, concatenation, etc.) to languages within that class results in a language that is also within the same class.
- Closure refers to some operation on a language, resulting in a new language that is of the same "type" as originally operated on i.e., regular. Regular languages are closed under the following operations:

Consider that L and M are regular languages

1. **Kleen Closure:** RS is a regular expression whose language is L, M. R* is a regular expression whose language is L*.

2. **Positive closure:** RS is a regular expression whose language is L, M. $\mathrm{R}+$ $R+$ is a regular expression whose language is $\mathrm{L}+$ $L+$.

3. **Complement:** The complement of a language L (with respect to an alphabet E such that E^* contains L) is E^* –L. Since E^* is surely regular, the complement of a regular language is always regular.

4. **Reverse Operator:** Given language L, L^R is the set of strings whose reversal is in L. Example: L = {0, 01, 100}; L^R ={0, 10, 001}.

   Proof: Let E be a regular expression for L. We show how to reverse E, to provide a regular expression E^R for L^R.

5. **Union:** Let L and M be the languages of regular expressions R and S, respectively. Then R+S is a regular expression whose language is(L U M).

6. **Intersection:** Let L and M be the languages of regular expressions R and S, respectively then it a regular expression whose language is L intersection M.

   proof: Let A and B be DFA's whose languages are L and M, respectively. Construct C, the product automaton of A and B make the final states of C be the pairs consisting of final states of both A and B.

7. **Set Difference operator:** If L and M are regular languages, then so is L – M = strings in L but not M.

   Proof: Let A and B be DFA's whose languages are L and M, respectively. Construct C, the product automaton of A and B make the final states of C be the pairs, where A-state is final but B-state is not.

8. **Homomorphism:** A homomorphism on an alphabet is a function that gives a string for each symbol in that alphabet. Example: h(0) = ab; h(1) = E . Extend to strings by h(a1…an) =h(a1)…h(an).

   Example: h(01010) = abababab. If L is a regular language, and h is a homomorphism on its alphabet, then h(L)= {h(w) | w is in L} is also a regular language.

   Proof: Let E be a regular expression for L. Apply h to each symbol in E. Language of resulting R, E is h(L).

9. **Inverse Homomorphism:** Let h be a homomorphism and L a language whose alphabet is the output language of h. $\mathrm{h}{-}1$ $h{-}1$ (L) = {w | h(w) is in L}.

Note: There are few more properties like symmetric difference operator, prefix operator, substitution which are closed under closure properties of regular language.

Decision Properties: Approximately all the properties are decidable in case of finite automaton.

```
(i) Emptiness
(ii) Non-emptiness
(iii) Finiteness
(iv) Infiniteness
(v) Membership
(vi) Equality
```

These are explained as following below. **(i)** Emptiness and Non-emptiness**:**

- **Step-1:** select the state that cannot be reached from the initial states & delete them (remove unreachable states).
- **Step 2:** if the resulting machine contains at least one final states, so then the finite automata accepts the non-empty language.
- **Step 3:** if the resulting machine is free from final state, then finite automata accepts empty language.
    - **Step-1:** select the state that cannot be reached from the initial state & delete them (remove unreachable states).
    - **Step-2:** select the state from which we cannot reach the final state & delete them (remove dead states).

- **Step-3:** if the resulting machine contains loops or cycles then the finite automata accepts infinite language.
- **Step-4:** if the resulting machine do not contain loops or cycles then the finite automata accepts finite language.

Operation	Closed (Regular Languages)
Union	Yes
Intersection	Yes
Set Difference	Yes
Complement	Yes
Concatenation	Yes
Kleene Star	Yes
Kleene Plus	Yes
Reversal	Yes
Homomorphism	Yes
Inverse Homomorphism	Yes
Substitution	Yes

Operation	Closed (Regular Languages)
Intersection with Regular Language	Yes
Union with Regular Language	Yes
Subset	No
Infinite Union	No